

Model order reduction for a transient reaction-convection-diffusion problem using the Laplace transform

Keyvan Attarian^{1,2} and Amine Ammar^{3,2}

¹Ecole polytechnique, Institut Polytechnique de Paris

²DesCartes Program, WP8, CNRS@CREATE

³ENSAM

Printed June 8, 2026

Abstract

Model order reduction for parametric transient partial differential equations (PDEs) typically suffers from the high computational cost of predicting long time horizons. To alleviate this, we propose a surrogate modeling strategy that leverages the Laplace transform to map the time domain into a highly compressed set of complex frequency fields. We systematically compare two non-linear compression approaches using autoencoders: Spatial Laplace Autoencoders (SLAE), which compress individual frequency frames, and Latent Laplace Autoencoders (LLAE), which apply the transform directly within a low-dimensional latent space. Our results demonstrate that end-to-end training of the surrogate models, combined with a lightweight UNet residual corrector to mitigate spectral truncation artifacts, significantly improves prediction accuracy. The best surrogate model achieves a relative L^2 error of ???%, compared to ???% for the linear SVD baseline, while providing a ??? $\times$ speedup over the high-fidelity solver.

Contents

1	Introduction	2
2	Methodology	3
3	Results	12
4	Comparison and Discussion	17
	Appendices	17

1 Introduction

High-fidelity simulations of parametric transient partial differential equations (PDEs), such as those governing combustion, reactive transport, and atmospheric dispersion, rely on computationally expensive numerical solvers. In many-query contexts like optimization, uncertainty quantification, real-time control, and digital twins, the complete spatiotemporal field $U(x, t; \mu)$ must be evaluated for thousands of parameter values μ [5, 17]. The prohibitive cost of these repeated evaluations motivates the development of surrogate models capable of reproducing the full dynamics in milliseconds. Applications such as air quality and pollution forecasting concretely illustrate this need: global physical models are accurate but unusable in real-time without approximation [9, 3].

Classical model order reduction (MOR) methods [10], such as Reduced Basis (RB) and Proper Orthogonal Decomposition (POD) combined with interpolation, construct a low-dimensional linear subspace by projecting high-fidelity solutions onto spatial modes. These approaches are theoretically well-founded and highly effective for smooth dynamics. However, they suffer from a reconstruction floor linked to the linearity of the basis: for problems with sharp transients, localized structures, or strongly non-linear dynamics (such as convection-diffusion-reaction equations), the number of required modes explodes. The treatment of time in these methods is either sequential (time-stepping on the projected ROM) or space-time (variational), both of which impose a computational cost that grows with the temporal horizon [6, 12, 20]. The transition toward non-linear manifolds becomes necessary as linear techniques struggle [4]. To overcome the linear floor, two main families of data-driven approaches have emerged alongside the advent of Physics-Informed Neural Networks [22]. On one hand, deep learning-based ROMs (DL-ROMs) [13, 4] jointly learn a non-linear manifold and reduced dynamics via convolutional autoencoders, demonstrating significant gains over linear ROMs for parametric non-linear PDEs. On the other hand, neural operators directly learn mappings between function spaces (i.e., field-to-field mappings). Notable examples include DeepONet [15] and its POD-based extensions [16], and the Fourier Neural Operator (FNO) [14], which employs convolution in the Fourier space. Because they are inherently designed to map an input function to an output function, they are not naturally suited for direct parameter-to-field predictions where the input is a low-dimensional scalar vector. Furthermore, these approaches treat time as an additional continuous dimension, imposing a high memory and computational cost for long sequences and limiting their ability to capture long-horizon transients without dense training data. Parameter-conditioned U-Nets [11] represent a lighter alternative for 2D convection-diffusion-reaction PDEs, yet they still operate directly in the time domain.

Spectral decompositions offer an alternative perspective for transient PDEs. FNO [14] parameterizes the integral kernel directly in the Fourier space, giving it remarkable efficiency on periodic or quasi-stationary problems. However, the Fourier transform implicitly assumes periodicity and is poorly suited for transient signals: it struggles to represent the natural exponential growth or decay of solutions and suffers from Gibbs artifacts in the presence of temporal discontinuities. The Laplace transform $s = \gamma + i\omega$ generalizes Fourier by adding a real decay axis γ that naturally encodes transients, exponential stability, and non-periodic regimes. Each frequency field $\hat{U}(s_k)$ is a static complex spatial field, structurally simpler to compress than an entire temporal sequence. The Laplace Neural Operator (LNO) [2] illustrates this advantage: by exploiting the pole-residual relationship in the Laplace domain, LNO surpasses FNO on transient responses and non-periodic signals for benchmark ODEs and PDEs. Regarding linear

reduction, recent works exploit the Laplace domain to construct reduced bases: Reyes [23] shows via the Paley-Wiener theorem that the Laplace domain produces exponentially accurate POD bases for evolution problems, and Goeckner et al. [7] extend this to parametric PDEs via contour integral methods. Nonetheless, these approaches remain fundamentally linear. The remaining gap is to replace this linear compression with non-linear autoencoders jointly learned with a parametric surrogate, enabling end-to-end differentiable training thanks to the exact invertibility of the transform [8].

We consider the surrogate modeling of parametric transient PDEs, where the goal is to learn the mapping $\boldsymbol{\mu} \mapsto (U(t))_{t=1}^{N_t}$ without running the numerical solver. Here $\boldsymbol{\mu} \in \mathbb{R}^p$ is a vector of physical parameters, $U(t) \in \mathbb{R}^{N_x}$ is the discretized spatial field at discrete time step $t \in \{1, \dots, N_t\}$, N_x is the number of spatial nodes, N_t the number of time steps, and K the number of Laplace frequency samples used in the spectral decomposition. This paper proposes to exploit the Laplace transform as a spectral compression tool for the spatiotemporal field, combined with non-linear autoencoders. Two AE strategies are systematically compared: compression in the physical domain frequency-by-frequency (SLAE), and compression in the temporal latent space prior to transformation (LLAE). For each, two surrogate strategies are examined: direct embedding prediction and SVD dimensionality reduction. End-to-end training is made possible by the differentiability of the inversion operator. A lightweight UNet residual corrector addresses Gibbs artifacts stemming from spectral truncation. Our contributions are threefold: (1) Formulation and comparison of two AE architectures in the Laplace domain (SLAE vs LLAE) with direct and SVD-compressed surrogate variants; (2) Demonstration that end-to-end fine-tuning of the decoder is indispensable for closing the gap between standalone AE performance and full surrogate accuracy; (3) Introduction of a differentiable UNet residual corrector for mitigating spectral artifacts.

The remainder of this paper is organized as follows. Section 2 describes the methodology, detailing the Laplace transform, surrogate architectures, and the end-to-end training pipeline. Section 3 presents the numerical results and performance comparisons. Finally, Section 4 provides a discussion and concludes the work. The appendices detail the governing equations and architectural hyperparameters.

2 Methodology

2.1 Laplace Transform and Inversion

Throughout this section, $u(t) \in \mathbb{R}$ denotes the field value at a single fixed spatial location (lowercase = one pixel); for the full spatial field $U(t) \in \mathbb{R}^{N_x}$ (uppercase = full grid of N_x nodes), all operations below are applied independently at each grid point. The forward Laplace transform is evaluated at discrete frequencies $s = \gamma + i\omega$ using a temporal quadrature rule (e.g., trapezoidal rule with weights w_n):

$$(1) \quad \hat{u}(s) = \int_0^T u(t) e^{-st} dt \approx \Delta t \sum_{n=0}^{N_t-1} w_n u(t_n) e^{-st_n}$$

Since $u(t)$ is strictly real-valued, its Laplace transform satisfies the conjugate symmetry property $\hat{u}(\bar{s}) = \hat{u}(s)$. Consequently, we restrict our numerical evaluation to frequencies with non-negative imaginary parts $\omega_k \geq 0$, which halves the forward evaluation cost while providing the full spectral information required for inversion.

Regularized Inversion (Tikhonov). Let $\mathbf{F} \in \mathbb{C}^{K \times N_t}$ be the forward evaluation matrix, with $\mathbf{F}_{k,n} = \Delta t w_n e^{-s_k t_n}$, and let $\hat{\mathbf{u}} \in \mathbb{C}^K$, $\mathbf{u} \in \mathbb{R}^{N_t}$ be the vectors of Laplace coefficients and temporal values. Taking advantage of the real-valued constraint on $\mathbf{u}$, the reconstruction is formulated as a regularized complex least-squares problem:

$$(2) \quad \mathbf{u}^* = \arg \min_{\mathbf{u} \in \mathbb{R}^{N_t}} \|\mathbf{F}\mathbf{u} - \hat{\mathbf{u}}\|_{\mathbb{C}^K}^2 + \alpha_t \|\mathbf{D}_t \mathbf{u}\|_2^2 + \lambda \|\mathbf{u}\|_2^2$$

where $\mathbf{D}_t \in \mathbb{R}^{(N_t-1) \times N_t}$ is the first-difference operator penalizing temporal roughness, and $\lambda > 0$ is a Tikhonov ridge regularization parameter. The corresponding optimality condition (normal equations) yields a real symmetric linear system:

$$(3) \quad \left(\Re(\mathbf{F}^H \mathbf{F}) + \alpha_t \mathbf{D}_t^T \mathbf{D}_t + \lambda \mathbf{I} \right) \mathbf{u}^* = \Re(\mathbf{F}^H \hat{\mathbf{u}})$$

By solving this system, the conjugate frequencies are implicitly accounted for through the real-part operator $\Re(\cdot)$, ensuring a real-valued temporal reconstruction. Since the inversion is implemented via a linear solve (e.g., `torch.linalg.solve`), the operator is fully differentiable and compatible with gradient-based training. Crucially, this formulation holds for any set of frequencies (s_k) , with no constraint on their distribution in the complex plane.

Choice of contour: truncated Bromwich. A key design choice is the set of evaluation frequencies $(s_1, \dots, s_K)$. We adopt throughout the *truncated Bromwich contour*: a vertical line in the complex plane at $\Re(s) = \gamma$, with imaginary parts equal to the first K discrete Fourier frequencies,

$$(4) \quad s_k = \gamma + i \frac{2\pi k}{N_t \Delta t}, \quad k = 0, \dots, K-1,$$

where $\gamma > 0$ is the *damping parameter* and $K \leq \lfloor N_t/2 \rfloor + 1$ controls the spectral truncation. With this choice, the forward sum (1) reduces to a truncated DFT of the exponentially damped sequence:

$$(5) \quad \hat{u}(s_k) = \text{FFT} \left\{ u(t_n) \Delta t w_n e^{-\gamma t_n} \right\}_k, \quad k = 0, \dots, K-1,$$

making the forward pass computationally efficient. Since $K < \lfloor N_t/2 \rfloor + 1$, the spectrum is incomplete and we rely on the Tikhonov inversion (2)–(3) in all cases. Restricting to the Bromwich structure, however, imposes that all poles share the same real part γ and lie on a regular frequency grid — a strong constraint that may be suboptimal. Since the Tikhonov inversion handles any set of frequencies, one can in principle place the poles freely in the complex plane to better capture the signal’s dynamics. This is the motivation for Section 2.5, which discusses how to optimize the contour beyond the Bromwich family.

2.2 SVD Baseline in the Laplace Domain

A linear baseline approach applies a truncated Singular Value Decomposition (SVD) independently to each Laplace frequency s_k . The complex spatial field $\hat{U}(s_k)$ is separated into its real and imaginary components, which are then projected onto pre-computed, fixed spatial bases $\mathbf{V}^{(k,r)}$ and $\mathbf{V}^{(k,i)}$:

$$(6) \quad \Re(\hat{U}(s_k)) \approx \sum_{j=1}^{k_{\text{svd}}} c_j^{(k,r)} \mathbf{V}_j^{(k,r)}, \quad \Im(\hat{U}(s_k)) \approx \sum_{j=1}^{k_{\text{svd}}} c_j^{(k,i)} \mathbf{V}_j^{(k,i)}$$

For each frequency k , dedicated Multilayer Perceptrons (MLPs), parameterized by weights θ , act as surrogates to predict the normalised coefficients $\mathbf{c}^{(k,r)}$ and $\mathbf{c}^{(k,i)}$ directly from the physical parameters μ . Specifically, independent MLPs are employed for the real and imaginary parts, governed by the following mappings:

$$(7) \quad \mathbf{c}^{(k,r)} = f_{\theta}^{(k,r)}(\mu), \quad \mathbf{c}^{(k,i)} = f_{\theta}^{(k,i)}(\mu)$$

where f_{θ} denotes the parametrized neural network function. These MLPs are trained to minimize the mean squared error (MSE) between the predicted and true SVD projection coefficients:

$$(8) \quad \mathcal{L}_{\text{SVD}}^{(k)} = \left\| f_{\theta}^{(k,r)}(\mu) - \mathbf{c}^{(k,r)} \right\|_2^2 + \left\| f_{\theta}^{(k,i)}(\mu) - \mathbf{c}^{(k,i)} \right\|_2^2$$

The time series $U(t)$ is subsequently reconstructed by multiplying the predicted coefficients with their respective bases, combining them into a complex field, and applying the inverse Laplace transform. However, this approach is fundamentally constrained by the irreducible truncation error of the linear spatial bases, particularly at higher frequencies where the spatial modes exhibit complex, highly localized structures.

2.3 Autoencoders in the Laplace Domain

To design non-linear reduced-order models, we investigate and compare two alternative autoencoder (AE) strategies:

- **Spatial Laplace AE (SLAE):** The Laplace transform is computed in the physical domain, and a frequency-conditioned autoencoder compresses and reconstructs each complex frequency frame independently.
- **Latent Laplace AE (LLAE):** A time-conditioned autoencoder first compresses the physical time-series frames into a low-dimensional latent space, and the Laplace transform is subsequently applied to the temporal latent codes.

In both configurations, we incorporate an L_2 “ridge” regularization penalty on the latent representations. This constraint is crucial as it enforces a sufficiently regular and smooth latent space structure, which subsequently makes it much easier for the surrogate model to learn the mapping from the physical parameters μ to the latent codes z (as detailed in Section 2.4).

Both architectures share common conditioning components: a sinusoidal coordinate encoding and Feature-wise Linear Modulation (FiLM). Below, we describe these mechanisms in a general manner using a coordinate $\tau \in [0, 1]$ that represents either the frequency ratio $f_k = k/(N_{\text{half}} - 1)$ in the spatial approach, or the normalised time step t/T in the latent approach.

Sinusoidal Coordinate Encoding. The coordinate $\tau \in [0, 1]$ is embedded via sinusoidal basis functions inspired by NeRF positional encoding [18]:

$$(9) \quad \mathbf{e}(\tau) = \text{MLP}\left(\left[\sin(2^i \pi \tau), \cos(2^i \pi \tau)\right]_{i=0}^{L-1}\right), \quad L = 8.$$

A plain scalar τ would conflate nearby coordinates and fail to capture sharp transitions in either the frequency or time domains. The multi-scale sinusoidal basis explicitly represents τ at L different resolutions, providing the network with a rich, non-saturating signal that allows fine discrimination across the entire range.

FiLM Conditioning. To share encoder/decoder weights across all coordinates (frequencies or time steps) while adapting to each specific configuration, the FiLM mechanism [21] is employed. At each convolutional layer ℓ , the coordinate embedding $\mathbf{e}(\tau)$ is projected to a channel-wise gain $\alpha_\ell \in \mathbb{R}^{C_\ell}$ and bias $\delta_\ell \in \mathbb{R}^{C_\ell}$, modulating the activations element-wise:

$$(10) \quad \mathbf{x}_\ell \leftarrow (\mathbf{x}_\ell \odot (1 + \alpha_\ell(\mathbf{e}(\tau)))) + \delta_\ell(\mathbf{e}(\tau)).$$

The residual formulation $(1 + \alpha_\ell)$ initializes the modulation close to the identity, which stabilizes early training. All coordinates share the same convolutional weights, and the FiLM parameters act as a lightweight per-coordinate adapter at each layer, enabling a single autoencoder to process the full spectrum or temporal sequence without training separate model copies.

2.3.1 Spatial Laplace AE (SLAE)

In SLAE, the Laplace transform operates directly on the time-domain spatial pixels, transforming the sequence of 2D grids $U(t)$ into K complex-valued frequency frames $\hat{U}(s_k)$. A frequency-conditioned encoder (FiLM-modulated convolutional network) compresses each 2-channel (real and imaginary) frame independently into a real latent vector $z(s_k) \in \mathbb{R}^{d_z}$:

$$(11) \quad U(t) \xrightarrow{\mathcal{L}_{\text{pixel-wise}}} \hat{U}(s_k) \xrightarrow{\text{Encoder}(f_k)} z(s_k) \xrightarrow{\text{Decoder}(f_k)} \tilde{U}(s_k) \xrightarrow{\mathcal{L}^{-1}} U_{\text{rec}}(t)$$

The corresponding decoder upsamples $z(s_k)$ back to the spatial field, with bifurcated output heads for the real and imaginary components to prevent cross-talk. The spatial autoencoder is trained by minimizing a loss function comprising the spatial reconstruction error in the Laplace domain and L_2 regularization on the latent representations:

$$(12) \quad \mathcal{L}_{\text{SLAE}} = \|\tilde{U}(s_k) - \hat{U}(s_k)\|_{\mathbb{C}^{N_x}}^2 + \beta \|z(s_k)\|_2^2$$

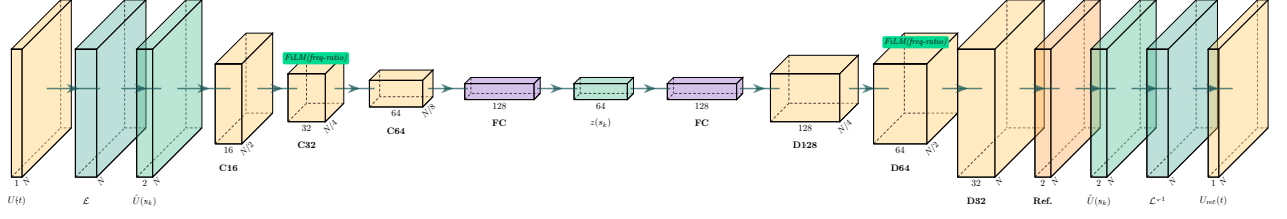
where $\|\cdot\|_{\mathbb{C}^{N_x}}^2$ is the squared L_2 norm over the N_x spatial nodes (real and imaginary parts combined), and β is a regularization hyperparameter.

2.3.2 Latent Laplace AE (LLAE)

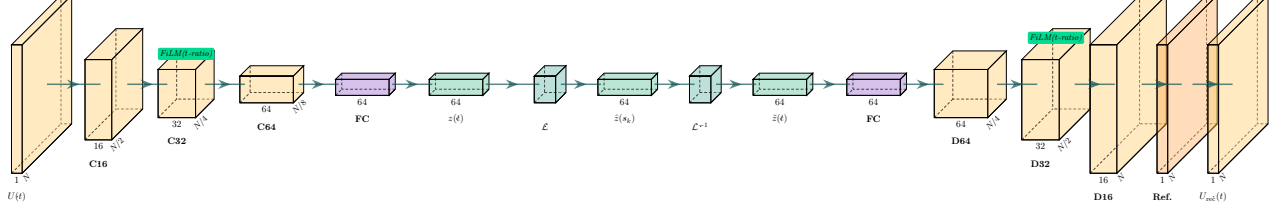
Alternatively, LLAE applies the transform within the low-dimensional latent space. The single-channel physical fields $U(t)$ are first encoded frame-by-frame by a similar convolutional encoder into latent codes $z(t) \in \mathbb{R}^{d_z}$, but conditioned on the time ratio t/T . The forward Laplace transform is then applied along the time dimension of the latent sequence:

$$(13) \quad U(t) \xrightarrow{\text{Encoder}(t)} z(t) \xrightarrow{\mathcal{L}} \hat{z}(s_k) \xrightarrow{\mathcal{L}^{-1}} \tilde{z}(t) \xrightarrow{\text{Decoder}(t)} U_{\text{rec}}(t)$$

This formulation fundamentally reduces the computational overhead of the transform and regularises the dynamics by enforcing temporal consistency within the latent space. However, it introduces a significant computational drawback: LLAE requires passing all N_t physical time frames through the autoencoder (both encoding and decoding) during training and inference, whereas SLAE only processes the K frequency frames. Since $N_t \gg N_L$, the LLAE approach is computationally much heavier. Consequently, under a fixed computational budget, we are constrained in terms of the maximum capacity (or



(a) SLAE: Laplace applied pixel-wise to the spatial field (large teal blocks, height $\propto N$). The decoder starts at 128 channels (D128), reflecting the 2-channel Re/Im reconstruction.



(b) LLAE: Laplace applied only in the compact $d_z = 64$ latent space (small teal blocks). The decoder starts at 64 channels (D64), visibly narrower than SLAE at the same scale.

Figure 1: SLAE (a) and LLAE (b) autoencoder pipelines at the same scale. Box widths are proportional to channel count ($16 \rightarrow 5$, $32 \rightarrow 7$, $64 \rightarrow 10$, $128 \rightarrow 13$) and heights to spatial resolution. The SLAE decoder ($D128 \rightarrow D64 \rightarrow D32$) is visibly wider than the LLAE decoder ($D64 \rightarrow D32 \rightarrow D16$).

volume) of the autoencoder that can be deployed. Specifically, the loss function is defined as:

$$\begin{aligned}
 \mathcal{L}_{\text{LLAE}} = & \frac{1}{N_t} \sum_{n=0}^{N_t-1} \|U_{\text{rec}}(t_n) - U(t_n)\|_2^2 \\
 & + \beta_{\text{latent}} \frac{1}{N_t} \sum_{n=0}^{N_t-1} \|\tilde{z}(t_n) - z(t_n)\|_2^2 \\
 & + \beta \frac{1}{K} \sum_{k=0}^{K-1} \|\hat{z}(s_k)\|_{\mathbb{C}^{d_z}}^2
 \end{aligned}
 \tag{14}$$

which combines the physical-time spatial reconstruction error, the latent reconstruction error between the target latent codes $z(t_n)$ and the reconstructed codes $\tilde{z}(t_n)$, and an L_2 regularization penalty on the complex Laplace-domain latent representation $\hat{z}(s_k)$.

The two architectures are compared at the same scale in Figure 1: box widths are proportional to channel count, so the wider SLAE decoder ($D128 \rightarrow D64 \rightarrow D32$) and the tall spatial Laplace blocks are immediately visible against the more compact LLAE pipeline.

2.4 Surrogate Training

The surrogate model g_θ replaces the encoding steps by learning the direct mapping $\mu \mapsto w_k$, where w_k denotes the Laplace-domain latent at frequency s_k that feeds the decoder: $w_k = z(s_k) \in \mathbb{R}^{d_z}$ for SLAE, and $w_k = \hat{z}(s_k) \in \mathbb{C}^{d_z}$ for LLAE. Two strategies for constructing the surrogate target w_k are described below.

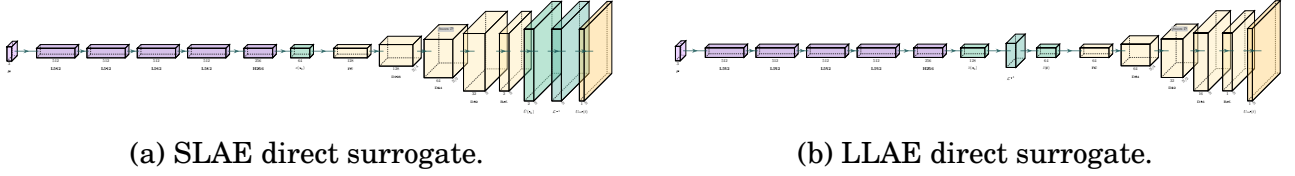


Figure 2: Direct embedding prediction. The FreqSurrogate trunk MLP (4 layers, hidden dim 512) maps μ to frequency-indexed latents, decoded by the AE decoder. Left: SLAE predicts real latents $z(s_k) \in \mathbb{R}^{d_z}$. Right: LLAE predicts complex latents $\hat{z}(s_k) \in \mathbb{C}^{d_z}$ (stored as $2d_z$ reals).

2.4.1 Direct Embedding Prediction

In the direct approach, the surrogate predicts the full latent vector at each index without any preliminary compression. For SLAE, the encoder produces frequency-domain latents $z(s_k) \in \mathbb{R}^{d_z}$ directly, and the pipeline is:

$$(15) \quad \mu \xrightarrow{g_\theta} z_{\text{pred}}(s_k) \xrightarrow{\text{Decoder}} \tilde{U}(s_k) \xrightarrow{\mathcal{L}^{-1}} U_{\text{rec}}(t)$$

For LLAE, the time-domain latent sequence $z(t) \in \mathbb{R}^{d_z}$ is first transformed to the Laplace domain, and the surrogate predicts those Laplace-domain codes directly:

$$(16) \quad \mu \xrightarrow{g_\theta} \hat{z}_{\text{pred}}(s_k) \xrightarrow{\mathcal{L}^{-1}} \tilde{z}_{\text{pred}}(t) \xrightarrow{\text{Decoder}} U_{\text{rec}}(t)$$

Both pipelines are illustrated in Figure 2.

2.4.2 SVD Dimensionality Reduction

To reduce the effective output dimension of the surrogate and simplify its learning task, one can apply a preliminary offline SVD compression to the latent sequence before training the surrogate, an approach inspired by [19]. Given the latent codes $\{e_i(\mu)\}_{i=1}^M$ collected over the training set (with $M = K$ for SLAE, $M = N_t$ for LLAE), a truncated SVD yields a projection matrix $V \in \mathbb{R}^{d_z \times d_{\text{SVD}}}$ (with $d_{\text{SVD}} \ll d_z$) such that $e_i \approx (e_i V) V^\top$. The reduced codes $G_i(\mu) = e_i(\mu) V \in \mathbb{R}^{d_{\text{SVD}}}$ become the surrogate targets, reducing the prediction problem from dimension d_z to d_{SVD} .

For SLAE (latents indexed by frequency k): the projection yields $G(s_k) = z(s_k) V$, and the surrogate predicts in the compressed frequency-domain space:

$$(17) \quad \mu \xrightarrow{g_\theta} G_{\text{pred}}(s_k) \xrightarrow{V^\top} z_{\text{pred}}(s_k) \xrightarrow{\text{Decoder}} \tilde{U}(s_k) \xrightarrow{\mathcal{L}^{-1}} U_{\text{rec}}(t)$$

For LLAE (latents indexed by time t): the projection yields $G(t) = z(t) V$, on which the Laplace transform is applied before surrogate prediction:

$$(18) \quad \mu \xrightarrow{g_\theta} \hat{G}_{\text{pred}}(s_k) \xrightarrow{\mathcal{L}^{-1}} \tilde{G}_{\text{pred}}(t) \xrightarrow{V^\top} \tilde{z}_{\text{pred}}(t) \xrightarrow{\text{Decoder}} U_{\text{rec}}(t)$$

The LLAE-SVD pipeline is illustrated in Figure 3.

In both strategies, the entire pipeline is trained end-to-end to minimise the total physical-time loss:

$$(19) \quad \mathcal{L}_{\text{total}} = \alpha_{\text{lat}} \|w_k^{\text{pred}} - w_k^{\text{true}}\|^2 + \alpha_{\text{spat}} \|U_{\text{rec}} - U\|^2$$

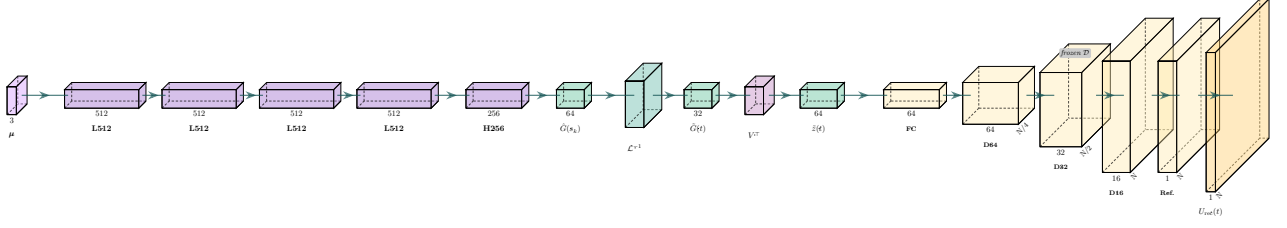


Figure 3: SVD surrogate pipeline (shown for LLAE). An offline SVD compresses the latent sequence to $G(t) = z(t)V \in \mathbb{R}^{d_{\text{SVD}}}$. The surrogate predicts Laplace-domain SVD codes $\hat{G}(s_k) \in \mathbb{C}^{d_{\text{SVD}}}$ (stored as $2d_{\text{SVD}}$ reals), which are inverted, back-projected via $V^\top$, and decoded by the AE decoder.

where w_k^{pred} and w_k^{true} denote the predicted and true Laplace-domain latents (either $z(s_k)$ or $\hat{G}(s_k)$ depending on the chosen strategy). End-to-end backpropagation allows gradients from the spatiotemporal MSE to flow through the decoder and the inverse transform, jointly updating the surrogate weights, the AE decoder, and, in the SVD variant, the projection basis V .

2.4.3 Tucker Decomposition in Latent Space

The SVD approach described above reduces the latent sequence along a single mode at a time: for LLAE it collapses the latent dimension d_z , leaving the time axis uncompressed; for SLAE it collapses d_z independently at each frequency, ignoring cross-frequency correlations. Tucker decomposition [?] generalises this by simultaneously factorising two modes of the latent tensor. Assembling the LLAE training latents into a three-way tensor $\mathcal{Z} \in \mathbb{R}^{N_{\text{train}} \times N_t \times d_z}$, a Tucker- (r_t, r_z) approximation decomposes each slice as

$$(20) \quad z_i(t) \approx U^{(t)} \mathcal{G}_i U^{(z)\top}, \quad U^{(t)} \in \mathbb{R}^{N_t \times r_t}, \quad U^{(z)} \in \mathbb{R}^{d_z \times r_z},$$

where $\mathcal{G}_i \in \mathbb{R}^{r_t \times r_z}$ is the simulation-specific core matrix and the factor matrices $U^{(t)}$, $U^{(z)}$ are shared across all training samples. The orthogonal factors are computed offline by the Higher-Order Orthogonal Iteration (HOOI) algorithm [?], which alternates truncated SVD steps along each mode until convergence to a stationary point of the Frobenius-norm approximation error. The surrogate then learns the mapping $\mu \mapsto \mathcal{G}(\mu) \in \mathbb{R}^{r_t \times r_z}$, whose dimensionality $r_t \cdot r_z$ can be substantially smaller than the SVD target $N_t \cdot d_{\text{SVD}}$ when temporal and latent correlations are jointly exploited. Reconstruction proceeds as $\tilde{z}(t) = U^{(t)} \mathcal{G}(\mu) U^{(z)\top}$, followed by the decoder. The Laplace transform can be applied either to $\tilde{z}(t)$ in the full latent space (analogously to LLAE) or directly to the time-mode projection $G(t) = \mathcal{G}(\mu) U^{(z)\top} \in \mathbb{R}^{r_t \times d_z}$ before inversion, offering an additional trade-off between compression ratio and spectral fidelity.

2.5 Choosing an Optimal Laplace Contour

The question of how to choose the integration contour $(s_1, \dots, s_K)$ arises for both SLAE and LLAE, but the two architectures admit fundamentally different strategies.

2.5.1 Learnable Contour for LLAE

In the LLAE setting, the Laplace transform is applied in the low-dimensional latent space: the surrogate directly predicts latent Laplace coefficients, and the complex frequencies s_k can be made learnable parameters of the model, optimized end-to-end with

the rest of the network. This is possible because both the forward quadrature (1) — a finite sum of complex exponentials $e^{-s_k t_n}$, smooth in s_k — and the Tikhonov inversion (3) — a linear solve whose coefficient matrix A depends smoothly on s_k , α_t , and λ — are fully differentiable with respect to the contour and regularization parameters.

This is precisely what is done in our LLAE implementation with `learnable_laplace=True`: both the real parts $\sigma_k = \text{Re}(s_k)$ and imaginary parts $\omega_k = \text{Im}(s_k)$ of each of the K poles are learned independently, along with the regularization hyperparameters α_t and λ (via their log-parameterisation). The poles are initialised on the truncated Bromwich contour (ω_k at the K first FFT frequencies, $\sigma_k = \gamma_{\text{init}}$ uniform).

2.5.2 Offline Contour Optimisation for SLAE

In contrast, making the SLAE contour learnable would require recomputing all N_t spatial forward passes per gradient step on the frequencies — negating its primary computational advantage over LLAE. Instead, we seek an *offline* procedure to select a good fixed contour before training SLAE.

We present a theoretical framework for this purpose. The goal is to decompose the total reconstruction error $\mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu})$ into a *bias* term (introduced by regularization and the finite contour) and a *variance* term (the surrogate error propagated through the inversion), and to jointly optimize the contour and regularization parameters by minimising an upper bound on the expected reconstruction error.

Formal Setup. Using the notation of Section 2.1, we fix a spatial location and work with scalar (per-pixel) quantities. Let $\mathbf{u}(\boldsymbol{\mu}) \in \mathbb{R}^{N_t}$ be the true temporal signal $(u(t_n))_n$ at that location, $\hat{\mathbf{u}}(\boldsymbol{\mu}) \in \mathbb{C}^K$ its exact Laplace coefficients $(\hat{u}(s_k))_k$, and $\tilde{\mathbf{u}}(\boldsymbol{\mu}) \in \mathbb{C}^K$ the surrogate-predicted coefficients (the per-pixel counterpart of the full-field $\tilde{U}(s_k)$ of Section 2.3.1). The regularised reconstructions from the true and predicted coefficients are, respectively,

$$(21) \quad \mathbf{u}^*(\boldsymbol{\mu}) = A^{-1} \Re \left(F^H \hat{\mathbf{u}}(\boldsymbol{\mu}) \right), \quad \mathbf{u}_{\text{rec}}(\boldsymbol{\mu}) = A^{-1} \Re \left(F^H \tilde{\mathbf{u}}(\boldsymbol{\mu}) \right),$$

where $A = \Re(F^H F) + \alpha_t D_t^T D_t + \lambda I$ is the normal-equation matrix of (3) and $L_{\text{reg}} = \alpha_t D_t^T D_t + \lambda I$. The unique existence of these solutions is guaranteed since $\lambda > 0$ and $D_t^T D_t$ is positive semi-definite, making $L_{\text{reg}} \succeq \lambda I$ and hence A symmetric positive definite (see Appendix D).

Bias-Variance Decomposition. We decompose the total reconstruction error by introducing $\mathbf{u}^*(\boldsymbol{\mu})$ and adding and subtracting it:

$$(22) \quad \begin{aligned} \mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu}) &= (\mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}^*(\boldsymbol{\mu})) + (\mathbf{u}^*(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu})) \\ &= (\mathbf{u}(\boldsymbol{\mu}) - A^{-1} \Re(F^H F) \mathbf{u}(\boldsymbol{\mu})) + A^{-1} \Re \left(F^H (\hat{\mathbf{u}}(\boldsymbol{\mu}) - \tilde{\mathbf{u}}(\boldsymbol{\mu})) \right). \end{aligned}$$

Using $A = \Re(F^H F) + L_{\text{reg}}$, the bias term simplifies to:

$$(23) \quad \mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}^*(\boldsymbol{\mu}) = A^{-1} \left(A - \Re(F^H F) \right) \mathbf{u}(\boldsymbol{\mu}) = A^{-1} L_{\text{reg}} \mathbf{u}(\boldsymbol{\mu}).$$

Taking norms in (22), applying the triangle inequality, and using $\|\Re(F^H \cdot)\| \leq \|F^H\| \|\cdot\|$ yields:

Theorem 2.1 (Bias–variance bound). *For any μ , the reconstruction error satisfies*

$$(24) \quad \|\mathbf{u}(\mu) - \mathbf{u}_{\text{rec}}(\mu)\| \leq \underbrace{\|A^{-1}L_{\text{reg}}\mathbf{u}(\mu)\|}_{\text{regularization bias}} + \underbrace{\|A^{-1}F^H\|}_{\text{amplification factor}} \cdot \underbrace{\|\hat{\mathbf{u}}(\mu) - \tilde{\mathbf{u}}(\mu)\|_{\mathbb{C}^K}}_{\text{surrogate error}}.$$

The regularization bias depends on the true field $\mathbf{u}(\mu)$ and L_{reg} , and is mitigated by the well-posedness of A^{-1} . Any surrogate prediction error $\hat{\mathbf{u}} - \tilde{\mathbf{u}}$ is magnified during reconstruction by the amplification factor $\|A^{-1}F^H\|$, which plays the role of a condition number for the inversion process.

Optimization Problem Formulation. The matrices F and L_{reg} , and consequently A , depend explicitly on the contour $(s_1, \dots, s_K)$, on λ , and on α_t . We aim to jointly optimize these parameters to minimise the expected mean reconstruction error $\mathbb{E}_\mu[\|\mathbf{u}(\mu) - \mathbf{u}_{\text{rec}}(\mu)\|]$. Since evaluating this true error directly is challenging without ground truth for every candidate contour, we treat the hyperparameters as learnable parameters and optimize a proxy loss $\mathcal{L}^{\text{proxy}}$ derived directly from the bound of Theorem 2.1:

$$(25) \quad \mathcal{L}^{\text{proxy}} = \mathcal{L}_{\text{bias}} + \mathcal{L}_{\text{var}}, \quad \mathcal{L}_{\text{bias}} = \mathbb{E}_\mu[\|\mathbf{u}(\mu) - \mathbf{u}^*(\mu)\|], \quad \mathcal{L}_{\text{var}} = \|A^{-1}F^H\| \cdot \epsilon_{\text{surr}},$$

where $\epsilon_{\text{surr}} = \mathbb{E}_\mu[\|\hat{\mathbf{u}}(\mu) - \tilde{\mathbf{u}}(\mu)\|_{\mathbb{C}^K}]$ is the expected surrogate error. By Jensen’s inequality ($\mathbb{E}[\sqrt{X}] \leq \sqrt{\mathbb{E}[X]}$), this expected norm is bounded by:

$$(26) \quad \epsilon_{\text{surr}} \leq \sqrt{\mathbb{E}_\mu[\|\hat{\mathbf{u}}(\mu) - \tilde{\mathbf{u}}(\mu)\|_{\mathbb{C}^K}^2]} = \sqrt{\sum_{k=1}^K \mathbb{E}_\mu[|\hat{u}_k(\mu) - \tilde{u}_k(\mu)|^2]},$$

Approximating the Surrogate Error via SVD Residual. Training a full neural surrogate for every proposed contour is computationally intractable. We therefore approximate the per-frequency expected surrogate error using the mean SVD residual. Here $\hat{U}(s_k; \mu) \in \mathbb{C}^{N_x}$ denotes the *full spatial* Laplace field at frequency s_k (the vector of per-pixel values $\hat{u}_k(\mu)$ across all N_x locations), as opposed to the scalar $\hat{u}_k(\mu)$ at a single location used above. For any specific frequency s_k , let $\mathbf{U}^{(k)} = [\hat{U}(s_k; \mu_1), \dots, \hat{U}(s_k; \mu_{N_{\text{train}}})] \in \mathbb{C}^{N_x \times N_{\text{train}}}$ be the dataset matrix formed by stacking the spatial Laplace fields across all training instances. The expected squared error of the optimal linear rank- d_z surrogate equals the mean squared column-wise error:

$$(27) \quad \mathbb{E}_\mu[\|\hat{U}(s_k; \mu) - \hat{U}^{\text{lin}}(s_k; \mu)\|^2] = \frac{1}{N_{\text{train}}} \|\mathbf{U}^{(k)} - \hat{\mathbf{U}}_{\text{lin}}^{(k)}\|_F^2.$$

The minimum of this squared Frobenius norm over all rank- d_z approximations is achieved by the truncated SVD and equals exactly the sum of the discarded squared singular values. The optimal linear baseline error is therefore:

$$(28) \quad \mathcal{E}_{\text{SVD}}(s_k) = \sqrt{\min_{\text{rank } \hat{\mathbf{U}} \leq d_z} \frac{1}{N_{\text{train}}} \|\mathbf{U}^{(k)} - \hat{\mathbf{U}}\|_F^2} = \sqrt{\frac{1}{N_{\text{train}}} \sum_{j>d_z} (\sigma_j^{(k)})^2},$$

where $\sigma_j^{(k)}$ are the singular values of $\mathbf{U}^{(k)}$. This establishes the fundamental lower bound for the expected error of any linear surrogate of latent dimension d_z . Since a non-linear AE with bottleneck d_z generalizes the linear SVD, the optimal AE error satisfies

$\epsilon_{\text{AE}}(s_k) \leq \mathcal{E}_{\text{SVD}}(s_k)$ (see Appendix D). In practice, the actual AE error may differ slightly due to optimisation challenges (local minima, architectural constraints), but for contour optimisation we are primarily interested in the *relative* behavior of ϵ_{surr} with respect to the contour rather than its absolute value. The SVD residual is fast and deterministic to compute for any candidate contour, making it highly suitable for iterative optimisation. To account for the enhanced expressivity of the non-linear AE over the linear SVD baseline, we introduce a scaling hyperparameter $\lambda_{\text{AE}} \geq 1$. The final proxy objective is:

$$(29) \quad \mathcal{L}^{\text{proxy}} = \frac{1}{N_{\text{train}}} \sum_{\mu} \|\mathbf{u}(\mu) - \mathbf{u}^*(\mu)\| + \frac{\|\mathbf{A}^{-1}\mathbf{F}^H\|}{\lambda_{\text{AE}}} \cdot \sqrt{\frac{1}{N_{\text{train}}} \sum_{k=1}^K \sum_{j>d_z} (\sigma_j^{(k)})^2}.$$

Minimizing (29) by gradient descent jointly optimizes the contour frequencies and the regularization parameters, trading bias against variance in a principled and fully differentiable manner.

3 Results

3.1 Problem Setting

We consider the emulation of the methane mass fraction Y in a two-dimensional parametric transient reacting flow. The transport equation is formulated as a convection-diffusion-reaction problem over a spatial domain $\Omega \subset \mathbb{R}^2$ and a time interval $t \in [0, T]$:

$$(30) \quad \frac{\partial(\rho Y)}{\partial t} + \nabla \cdot (\rho \mathbf{U} Y) - \nabla \cdot (\mu_{\text{eff}} \nabla Y) = \dot{\omega}_{\text{eff}}(Y; \mu)$$

where ρ is the fluid density, $\mathbf{U}$ is the velocity field, μ_{eff} is the effective diffusivity, and $\dot{\omega}_{\text{eff}}$ is the effective reaction rate. A detailed formulation of the complete system of governing equations is provided in Appendix B. The system is governed by a vector of three physical parameters $\mu = [k, \theta, C]^\top \in \mathbb{R}^3$, specifically representing:

- k : Diffusion parameter,
- θ : Injection angle,
- C : Injection rate (kg/m³/s).

The spatial domain is discretized into a uniform grid of $N_x = 128 \times 128 = 16,384$ nodes, and the temporal domain into $N_t = 150$ time steps with uniform step size $\Delta t = 1$ s (total duration $T = 150$ s). The parameter space has dimension $p = 3$ ($\mu \in \mathbb{R}^3$). The quantity of interest in the methodology notation is $U(t) \in \mathbb{R}^{N_x}$, the discretized spatial field of Y at time step $t \in \{1, \dots, N_t\}$; the full space-time field is a matrix in $\mathbb{R}^{N_x \times N_t}$. The number of Laplace frequency samples K is a model hyperparameter ($K = 16$ for both SLAE and LLAE; see Table 4). The dataset comprises 6480 training simulations and 1620 test simulations.

The performance of the models is evaluated quantitatively using the relative L^2 error over the held-out test set, defined as:

$$(31) \quad \epsilon_{\text{rel}} = \frac{\|Y_{\text{true}} - Y_{\text{pred}}\|_2}{\|Y_{\text{true}}\|_2}$$

We first characterize the autoencoder architectures in isolation, then evaluate the end-to-end surrogate models.

3.2 Autoencoder Performance

Table 1 reports the relative L^2 reconstruction error on the held-out test set (1620 simulations) for all trained AE checkpoints, evaluated on the physical field $U(t)$.

Model	d_z	K	γ	Median (%)	Mean (%)	p90 (%)
LLAE	128	16	0.01	2.92	3.15	4.38
	64	32	0.01	3.19	3.53	5.11
	64	16	0.01	3.20	3.47	5.04
	64	16	0.0	3.26	3.61	5.24
	64	16	0.001	3.32	3.65	5.16
	64	8	0.01	3.35	3.66	5.26
	32	16	0.01	3.43	3.85	5.60
SLAE	64	16	0.0	7.78	8.11	9.91
	64	16	0.001	7.82	8.27	10.36
	128	16	0.01	7.98	8.40	10.45
	64	32	0.01	8.34	9.27	13.43
	64	16	0.01	8.92	9.53	12.55
	32	16	0.01	10.76	11.14	14.76
	64	8	0.01	12.26	12.74	15.40

Table 1: Relative L^2 reconstruction error on the test set (1620 simulations). Best per family in bold.

Figure 4 summarises the reconstruction floor across all 14 checkpoints, sorted by median error with IQR error bars. LLAE consistently outperforms SLAE across all hyperparameter configurations, achieving a best median error of 2.92% ($d_z = 128$, $K = 16$) versus 7.78% for the best SLAE ($d_z = 64$, $K = 16$, $\gamma = 0$). This gap stems from the nature of the compression problem each architecture faces. SLAE must encode each complex frequency frame $\hat{U}(s_k) \in \mathbb{C}^{N_x}$ with $N_x = 128^2$ spatial points independently; this is a spatially rich, high-dimensional input where the convolutional bottleneck is genuinely constraining. LLAE instead encodes single-channel physical frames $U(t) \in \mathbb{R}^{N_x}$ that are smoother and more structured, before applying the Laplace transform in the compact d_z -dimensional latent space, a substantially easier per-frame task.

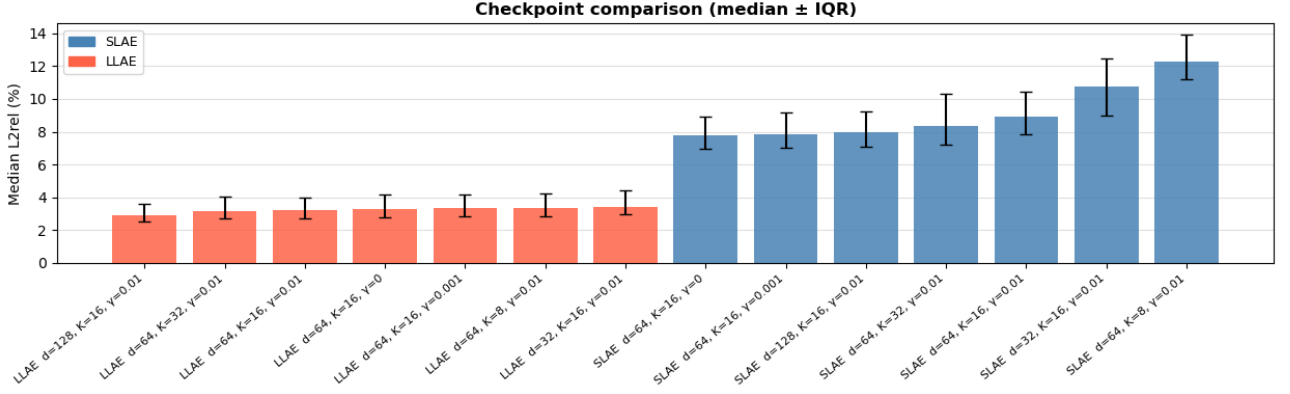


Figure 4: Reconstruction error for all 14 AE checkpoints, sorted by median relative L^2 error on the test set ($n = 1620$). Error bars indicate the interquartile range. LLAE variants (red) achieve consistently lower errors than SLAE variants (blue).

Figures 5 and 6 report ablation studies for each family, varying one hyperparameter at a time while holding the others fixed.

Within LLAE (Figure 5), increasing the latent dimension from $d_z = 32$ to $d_z = 128$ reduces the median error from 3.43% to 2.92%, indicating that the latent bottleneck has not yet saturated at $d_z = 64$. The number of Laplace frequencies K has little impact: errors at $K = 8, 16$, and 32 are 3.35%, 3.20%, and 3.19% respectively, suggesting that 8–16 frequencies already capture the dominant transient dynamics. The damping parameter γ is similarly insensitive in the range $[0, 0.01]$, consistent with the smooth energy envelope of the exponential decay at these timescales.

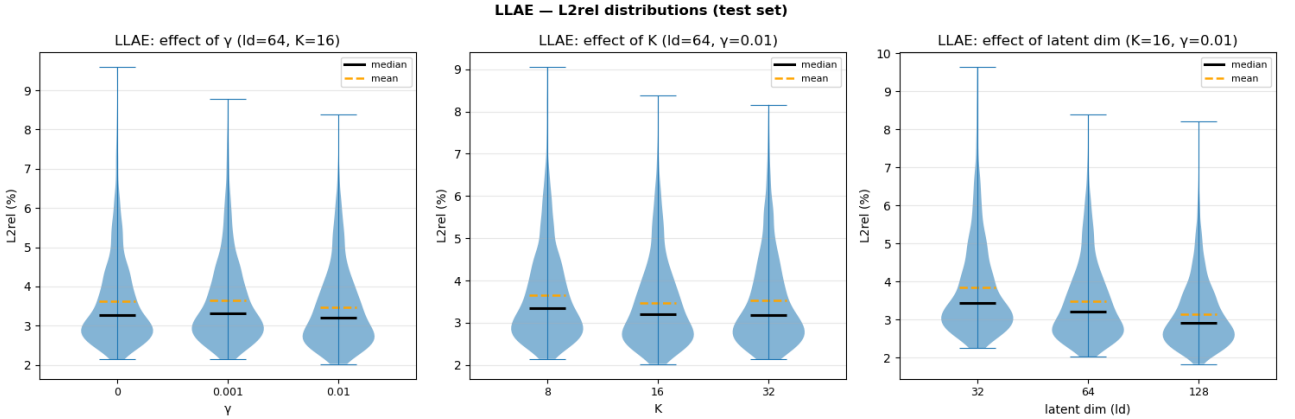


Figure 5: LLAE ablation study. From left to right: effect of latent dimension d_z ($K = 16$, $\gamma = 0.01$), number of Laplace frequencies K ($d_z = 64$, $\gamma = 0.01$), and damping parameter γ ($d_z = 64$, $K = 16$). Violin plots show the full error distribution; black line is the median, dashed orange line is the mean.

Within SLAE (Figure 6), errors increase sharply at low K ($K = 8$: 12.26%) and low d_z ($d_z = 32$: 10.76%), pointing to a harder compression problem where both the spectral and spatial bottlenecks are binding simultaneously. Increasing K to 32 does not improve reconstruction (8.34% vs. 7.78% at $K = 16$), likely because higher Laplace frequencies carry less spatial energy and expose the spatial compression bottleneck. The SLAE error distribution also shows a heavier right tail (higher p90) than LLAE, reflecting occasional failure cases where spatial compression degrades on outlier samples.

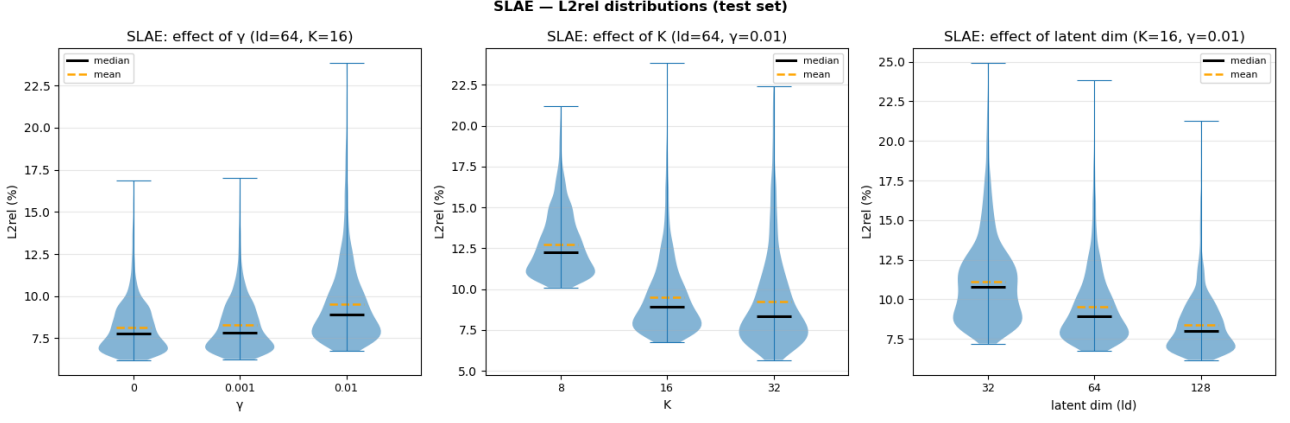


Figure 6: SLAE ablation study. Same layout as Figure 5 (fixed parameters: $K = 16$, $\gamma = 0.01 / d_z = 64$, $\gamma = 0.01 / d_z = 64$, $K = 16$). Note the larger y-axis range compared to LLAE.

For SLAE, Figure 7 shows the reconstruction error broken down per Laplace frequency k . The error is lowest at low frequencies ($k = 0-3$, where most signal energy is concentrated) and rises monotonically toward higher frequencies, confirming that the spatial compression bottleneck is more severe for the fine-scale, high-frequency content.

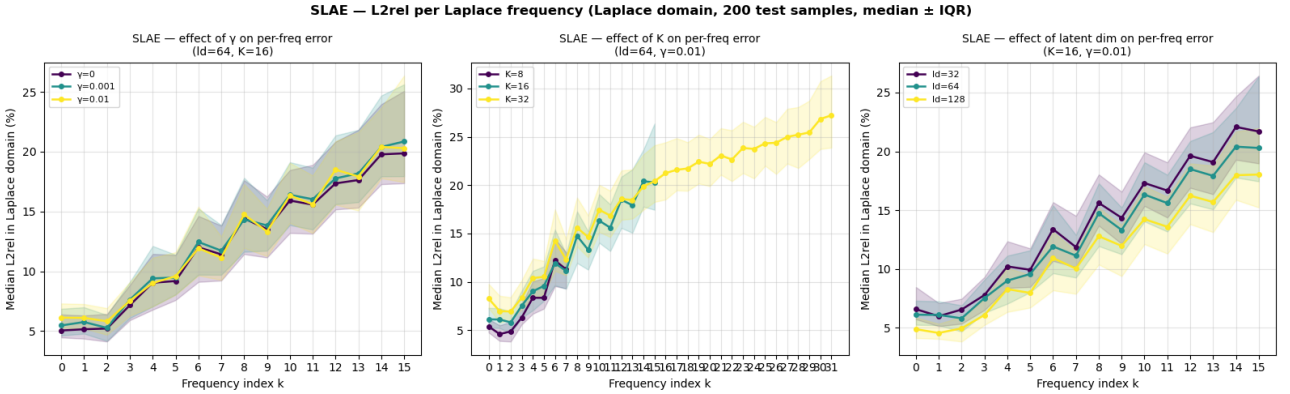


Figure 7: SLAE — per-frequency reconstruction error in the Laplace domain (median $\pm$ IQR, 200 test samples). Left: effect of γ ; centre: effect of K ; right: effect of d_z . Error increases monotonically with frequency index k , reflecting the concentration of signal energy at low frequencies.

Inference latency. Table 2 and Figure 8 report per-sample inference latency and GPU throughput measured with CUDA Events on an NVIDIA RTX 3090 (10 warm-up runs, 50 timed runs, batch size 1 for latency; max batch size for throughput). SLAE is approximately $2.4\times$ faster per sample than LLAE at $K = 16$ (8.0 ms vs. $\sim 19-20$ ms), because its inference path processes only K frequency frames through the decoder, whereas LLAE must decode all $N_t = 150$ time frames. At $K = 8$, SLAE latency drops to 5.5 ms. SLAE also requires far less GPU memory (112 MB at $K = 16$ vs. 525 MB for LLAE), enabling a $5\times$ larger maximum batch size.

Model	K	d_z	Latency (ms)	Throughput (s/s)	Peak GPU (MB)
SLAE	8	64	5.5 ± 0.2	224	56
SLAE	16	64	8.0 ± 0.1	134	112
SLAE	16	128	8.1 ± 0.1	133	112
SLAE	32	64	13.9 ± 0.4	74	224
LLAE	16	32	19.0 ± 1.1	56	525
LLAE	16	64	19.4–20.1	52–54	525
LLAE	16	128	19.0 ± 0.8	56	525

Table 2: Inference benchmark on an NVIDIA RTX 3090. Latency: batch size 1, 50 timed CUDA Event runs after 10 warm-up runs. Throughput at maximum GPU-filling batch size.

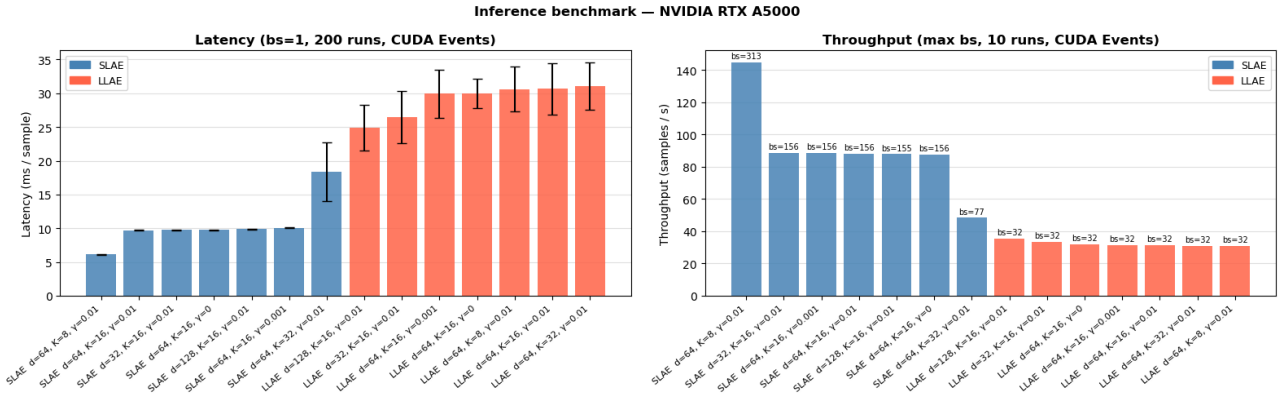


Figure 8: Inference benchmark on an NVIDIA RTX 3090. Left: per-sample latency (batch size 1, 50 CUDA Event runs). Right: throughput at maximum batch size. SLAE processes only K frequency frames per sample, yielding $2.4\times$ lower latency and $5\times$ larger feasible batch size than LLAE.

3.3 Surrogate Prediction Performance

Table 3 reports the relative L^2 error on the held-out test set (1620 simulations) for all trained surrogate checkpoints, evaluated on the physical field $U(t)$.

Model	d_z	K	γ	Median (%)	Mean (%)	p90 (%)
LLAE	128	16	0.01	???	???	???
	64	32	0.01	???	???	???
	64	16	0.01	???	???	???
	64	16	0.0	???	???	???
	64	16	0.001	???	???	???
	64	8	0.01	???	???	???
	32	16	0.01	???	???	???
SLAE	64	16	0.0	???	???	???
	64	16	0.001	???	???	???
	128	16	0.01	???	???	???
	64	32	0.01	???	???	???
	64	16	0.01	???	???	???
	32	16	0.01	???	???	???
	64	8	0.01	???	???	???

Table 3: Relative L^2 error on the test set (1620 simulations) for end-to-end surrogate models. Rows ordered as in Table 1. Best per family in bold.

4 Comparison and Discussion

4.1 Addressing Gibbs Oscillations: The Correction AE

Appendices

A Per-Checkpoint Error Distributions

Figure 9 shows the distribution of relative L^2 errors across the 1620 test simulations for each of the 14 AE checkpoints evaluated in Section 3.2. Dashed lines indicate the median (black) and mean (orange); the dotted red line marks the 90th percentile.

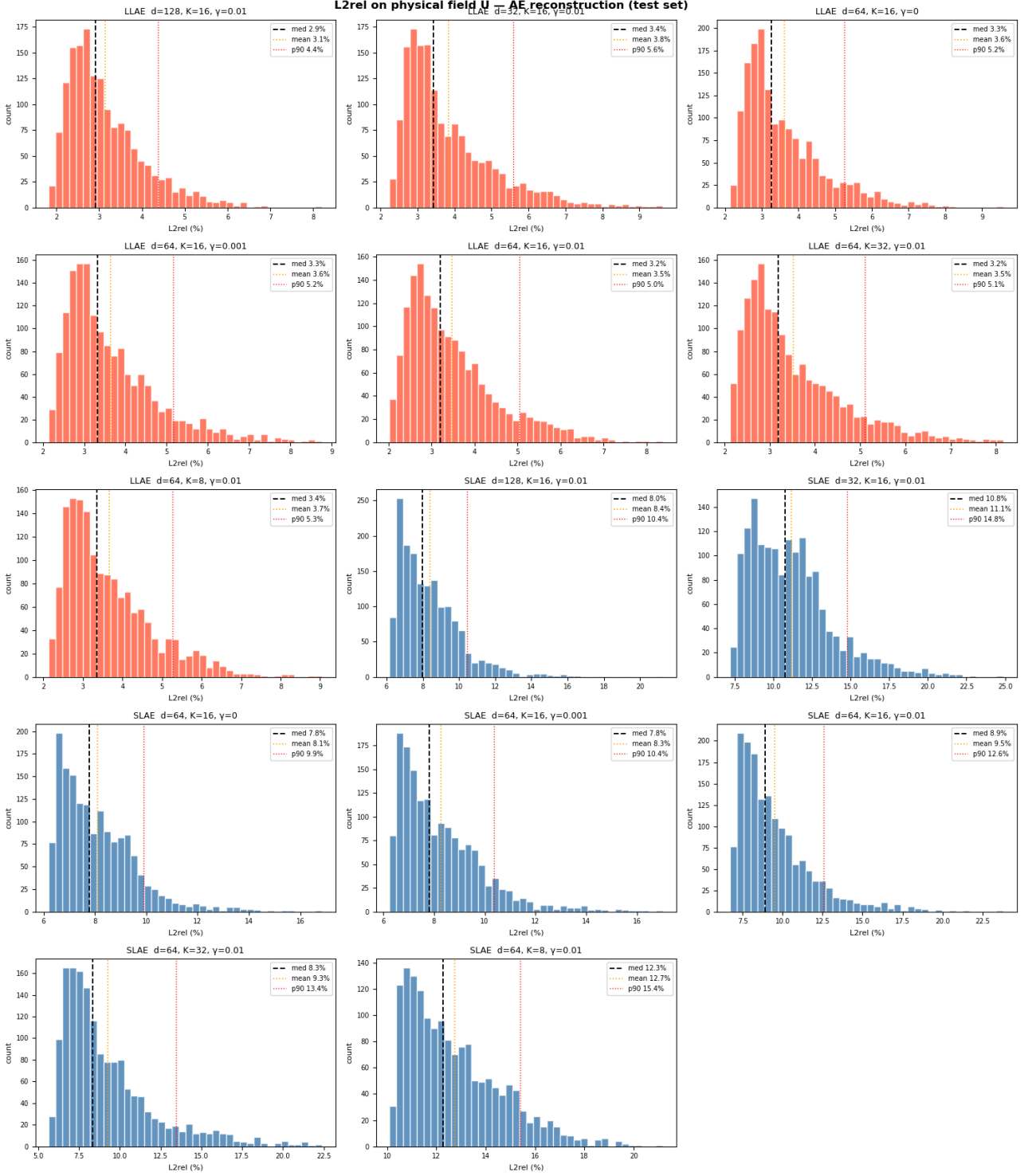


Figure 9: Per-checkpoint error histograms on the test set ($n = 1620$). Each panel corresponds to one checkpoint; SLAE in blue, LLAE in red. The dashed black line is the median, dashed orange the mean, dotted red the 90th percentile.

B Governing Equations

Our neural surrogate model is designed to emulate the transient dynamics of a reacting flow system. The physical system is governed by a set of coupled partial differential equations representing the conservation of mass, momentum, energy, and chemical species.

1. **Fluid Mechanics (Momentum):**

$$(32) \quad \frac{\partial(\rho \mathbf{U})}{\partial t} + \nabla \cdot (\rho \mathbf{U} \mathbf{U}) - \nabla \cdot \boldsymbol{\tau} = -\nabla p + \rho \mathbf{g}$$

2. **Chemical Species Transport:**

$$(33) \quad \frac{\partial(\rho Y_i)}{\partial t} + \nabla \cdot (\rho \mathbf{U} Y_i) - \nabla \cdot (\mu_{\text{eff}} \nabla Y_i) = \dot{\omega}_{i,\text{eff}}$$

under the constraint $\sum Y_i = 1$.

3. **Energy:**

$$(34) \quad \frac{\partial(\rho h)}{\partial t} + \nabla \cdot (\rho \mathbf{U} h) + \frac{\partial(\rho K)}{\partial t} + \nabla \cdot (\rho \mathbf{U} K) - \frac{\partial p}{\partial t} - \nabla \cdot (\alpha_{\text{eff}} \nabla h) = \dot{q}_{\text{chem}}$$

where $K = \frac{1}{2} |\mathbf{U}|^2$ is the kinetic energy.

4. **Combustion Model (PaSR):** The effective reaction rate is modulated by the Partially Stirred Reactor model:

$$(35) \quad \dot{\omega}_{i,\text{eff}} = \kappa \dot{\omega}_i(c^*, T^*), \quad \text{with} \quad \kappa = \frac{\tau_c}{\tau_c + \tau_{\text{mix}}}$$

where τ_c and τ_{mix} are the characteristic chemical and turbulent mixing times, respectively.

5. **Equation of State:** The ideal gas law closes the system:

$$(36) \quad p = \rho R T \quad \text{or} \quad \rho = \psi p \quad \text{with} \quad \psi = \frac{1}{RT}$$

C Architectural Details and Setup

All models were trained using the AdamW optimizer (weight decay 10^{-4}) with a learning rate of 5×10^{-4} for the autoencoders, 3×10^{-4} for the surrogate trunk, and 5×10^{-5} for the fine-tuned decoder, over 300 epochs. The autoencoders used a batch size of 256 (SLAE) or 16 (LLAE). Training was performed on a workstation equipped with an Intel(R) Xeon(R) w3-2435 CPU (62GB RAM) and an NVIDIA GeForce RTX 3090 GPU (24GB VRAM). Training data consists of 6480 simulations; the test set contains 1620 samples.

Table 4 summarises the hyperparameters of both autoencoder architectures.

Table 5 summarises the hyperparameters used for surrogate training (Phase 2).

Group	Hyperparameter	SLAE	LLAE
Architecture (baseline [†])	Latent dim d_z	64	64
	Frequencies K	16	16
	Sinusoidal levels L	8	8
	Contour γ	0.01	0.01
Regularization	Ridge β	10^{-2}	10^{-2}
	Latent roundtrip β_{latent}	—	0.1
	Tikhonov α_t	7×10^{-3} (eval)	7×10^{-3}
	Tikhonov λ	3×10^{-5} (eval)	3×10^{-5}
Training	Epochs	300	300
	Batch size	256	16
	Learning rate	5×10^{-4}	5×10^{-4}
	Early stopping patience	40	40

Table 4: Autoencoder hyperparameters. [†]Baseline values fixed when the other architectural parameters are swept: $d_z \in \{32, 64, 128\}$, $K \in \{8, 16, 32\}$, $\gamma \in \{0.0, 0.001, 0.01\}$.

Group	Hyperparameter	SLAE	LLAE
Architecture (fixed)	Trunk layers n_{trunk}	4	4
	Trunk hidden dim	512	512
	Head layers n_{head}	2	2
	Head hidden dim	256	256
Loss	Latent weight α_{lat}	1.0	1.0
	Spatial weight α_{spat}	1.0	1.0
	Tikhonov α_t	7×10^{-3}	7×10^{-3}
	Tikhonov λ	3×10^{-5}	3×10^{-5}
Training	Epochs	300	300
	Batch size	256	16
	$lr_{\text{surrogate}}$	3×10^{-4}	3×10^{-4}
	lr_{decoder}	5×10^{-5}	5×10^{-5}
	Weight decay	10^{-4}	10^{-4}
	Early stopping patience	40	40

Table 5: Surrogate training hyperparameters (Phase 2). MLP architecture and training setup are fixed across all runs; one surrogate is trained per AE checkpoint (14 total, sweeping d_z , K , γ as in Table 4). The AE decoder is fine-tuned at lr_{decoder} while the surrogate MLP is trained at $lr_{\text{surrogate}}$.

D Proofs for the Bias–Variance Contour Optimization

Proposition D.1 (Invertibility of A). *Since $\lambda > 0$ and $D_t^T D_t$ is positive semi-definite, $L_{\text{reg}} = \alpha_t D_t^T D_t + \lambda I$ is positive definite ($L_{\text{reg}} \succeq \lambda I$). Thus $A = \mathfrak{K}(F^H F) + L_{\text{reg}}$ is symmetric positive definite and hence invertible. The unique reconstructed solution is $u^*(\mu) = A^{-1} \mathfrak{K}(F^H \hat{u}(\mu))$.*

Proof of Theorem 2.1. Starting from decomposition (22) and applying the triangle inequality:

$$\|u(\mu) - u_{\text{rec}}(\mu)\| \leq \|u(\mu) - u^*(\mu)\| + \|u^*(\mu) - u_{\text{rec}}(\mu)\|.$$

The first term equals $\|A^{-1} L_{\text{reg}} u(\mu)\|$ by (23). For the second term, $u^*(\mu) - u_{\text{rec}}(\mu) = A^{-1} \mathfrak{K}(F^H (\hat{u}(\mu) - \tilde{u}(\mu)))$. Applying sub-multiplicativity of the operator norm and $\|\mathfrak{K}(M)\| \leq \|M\|$ yields $\|u^*(\mu) - u_{\text{rec}}(\mu)\| \leq \|A^{-1} F^H\| \cdot \|\hat{u}(\mu) - \tilde{u}(\mu)\|_{\mathbb{C}^K}$, which gives (24). $\square$

Proof ($\epsilon_{\text{AE}}(s_k) \leq \mathcal{E}_{\text{SVD}}(s_k)$). A linear autoencoder with bottleneck dimension d_z minimizing the mean squared reconstruction error recovers exactly the truncated SVD projection [1]. Since a nonlinear autoencoder with the same bottleneck dimension optimizes over a strictly larger class of functions (which includes all linear encoder–decoder pairs), its minimum reconstruction error satisfies $\epsilon_{\text{AE}}(s_k) \leq \mathcal{E}_{\text{SVD}}(s_k)$, as empirically confirmed in [24]. $\square$

References

- [1] Pierre Baldi and Kurt Hornik. Neural networks and principal component analysis: Learning from examples without local minima. *Neural Networks*, 2(1):53–58, 1989.
- [2] Qianying Cao, Somdatta Goswami, and George Em Karniadakis. LNO: Laplace neural operator for solving differential equations, 2023.
- [3] X. Fan, Y. Lin, B. Gong, and H. Li. Offline meteorology-pollution coupling global air pollution forecasting model with bilinear pooling, 2025.
- [4] Stefania Fresca, Luca Dedè, and Andrea Manzoni. A comprehensive deep learning-based approach to reduced order modeling of nonlinear time-dependent parametrized pdes. *Journal of Scientific Computing*, 87(2):61, 2021.
- [5] Matteo Giacomini and Pedro Díez. Surrogates for physics-based and data-driven modelling of parametric systems. 2026.
- [6] Silke Glas, Antonia Mayerhofer, and Karsten Urban. Two ways to treat time in reduced basis methods. In Peter Benner, Mario Ohlberger, Anthony Patera, Gianluigi Rozza, and Karsten Urban, editors, *Model Reduction of Parametrized Systems*, volume 17 of *MS&A*, pages 113–128. Springer, 2017.
- [7] et al. Goeckner. Model order reduction in contour integral methods for parametric pdes. 2022.
- [8] Nicola Guglielmi, María López-Fernández, and Giancarlo Nino. Numerical inverse laplace transform for convection-diffusion equations. 2020.
- [9] Q. Guo, M. Ren, S. Wu, et al. Applications of artificial intelligence in the field of air pollution: a bibliometric analysis. *Frontiers in Public Health*, 10:933665, 2022.

- [10] Jan S Hesthaven, Gianluigi Rozza, and Benjamin Stamm. *Certified Reduced Basis Methods for Parametrized Partial Differential Equations*. Springer, 2015.
- [11] Michael Urs Lars Kastor, Jan Rottmayer, Anna Hundertmark, and Nicolas Ralph Gauger. Parameter conditioned interpretable u-net surrogate model for data-driven predictions of convection-diffusion-reaction processes. 2026.
- [12] Toni Lassila, Andrea Manzoni, Alfio Quarteroni, and Gianluigi Rozza. Model order reduction in fluid dynamics: Challenges and perspectives. In Alfio Quarteroni and Gianluigi Rozza, editors, *Reduced Order Methods for Modeling and Computational Reduction*, volume 9 of *MS&A*, pages 235–274. Springer, 2014.
- [13] Kookjin Lee and Kevin T Carlberg. Model reduction of dynamical systems on non-linear manifolds using deep convolutional autoencoders. *Journal of Computational Physics*, 404:108973, 2020.
- [14] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2010.08895.
- [15] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3:218–229, 2021.
- [16] Lu Lu, Xuhui Meng, Shengze Cai, Zhiping Mao, Somdatta Goswami, Zhongqiang Zhang, and George Em Karniadakis. A comprehensive and fair comparison of two neural operators (with practical extensions) based on FAIR data. *Computer Methods in Applied Mechanics and Engineering*, 393:114778, 2022.
- [17] Swaroop Mallick and Monika Mittal. Ai-based model order reduction techniques: A survey. 2025.
- [18] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *European Conference on Computer Vision (ECCV)*, 2020.
- [19] Jad Mounayer, Sebastian Rodriguez, Chady Ghnatios, Charbel Farhat, and Francisco Chinesta. Rank reduction autoencoders. 2025.
- [20] Eric Parish, Masayuki Yano, Irina Tezaur, and Traian Iliescu. Stabilization of cdr roms. 2025.
- [21] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville. FiLM: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
- [22] Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
- [23] Ricardo Reyes. The frequency reduced-basis method: Reduced order models for time-dependent problems using the laplace transform. 2025.
- [24] Yasi Wang, Hongxun Yao, and Sicheng Zhao. Auto-encoder based dimensionality reduction. *Neurocomputing*, 184:232–242, 2016.